MySQL 读写性能测试

基于 Spring Data JPA 的 productdemoaop 应用

实验日期：2026 年 10 月 23 日

摘要

本实验在华为云四台服务器组成的 Docker Swarm 集群上，用 JMeter 对 productdemoaop 的商品查询与新建两个 RESTful API 进行压力测试，考察 Spring Data JPA 读写 MySQL 的性能。实验分两部分：一是按固定升压时间逐步提高线程数，比较读写响应时间（读取 5 档、写入 4 档，共 9 组）；二是让线程连续发送请求，测出读写各自的吞吐上限（读写各 4 组）。两部分共 17 组、175600 次请求，断言失败 0 次。读取吞吐上限约 1474–1567 次/秒，写入约 787–820 次/秒，瓶颈均为应用节点 CPU。同时用 Prometheus 记录 JVM 活跃线程、各节点 CPU 与内存使用率。

关键词：MySQL；Spring Data JPA；JMeter；Prometheus；性能测试

1 问题描述

数据库访问通常是 Web 应用的主要瓶颈之一。本实验在固定的应用、数据库和测试数据下，逐步增加并发线程数，测量“管理员查询产品”和“管理员新建产品”两个接口的响应时间、成功率和吞吐量，比较 JPA 读操作与写操作的性能差异，并结合监控数据分析负载在各节点上的分布。所有原始 JTL、JMeter 网页报告、监控数据和过程截图均随报告提交，供逐项核对。

2 实验分析

2.1 实验环境

节点	内网 IP	用途	主要软件
compiler	172.31.4.138	编译与压力测试（服务器 A）	JDK 21.0.2、Maven 3.9.16、JMeter 5.6.3
course-node2	172.31.8.71	应用服务（服务器 B）	Spring Boot 4.1.1、Spring Data JPA、Druid
course-node1	172.31.6.17	数据库（服务器 C）	MySQL 9.2.0
manager	172.31.13.176	Swarm 管理与监控（服务器 D）	Prometheus 3.7.3

四台服务器均为 Ubuntu 18.04.6、2 核 CPU、2 GiB 内存（系统可用约 1.8 GiB），通过私网互联。应用、数据库和 Prometheus 以 Docker Swarm 服务部署，并用节点标签固定到对应主机；每个节点运行 node-exporter。

应用 Tomcat 最大工作线程 300、最小空闲线程 100，Druid 连接池最大 300、最小空闲 100、初始 100；应用 JVM 未显式设置堆大小。JMeter 堆为 256–512 MiB、线程栈 256 KiB。指导书建议的 2–4 GiB 堆在本机内存下无法启动（JVM 报告无法分配 2 GiB 内存），因此按实际内存调小。JMeter 报告采样粒度设为 1000 ms。

2.2 实验方法

读取接口为 GET /shops/{shopId}/products/{id}，shopId 与 id 从 products.csv 中依次读取；写入接口为 POST /shops/1/products，商品名称、价格、重量、条码等字段由 JMX 中的随机函数生成。读取断言 HTTP 200 且返回 errno=0，写入断言 HTTP 201。JMeter 以非 GUI 模式运行，测试目标为应用服务器内网地址 172.31.8.71:8080，每组保存 JTL 原始数据并生成 HTML 报告。

操作	线程数	升压时间	每线程循环	组数 / 请求数
读取	1000、1050、1100、1150、1200	10 秒	2 次	5 组 / 11000 次
写入	1000、1100、1200、1300	10 秒	1 次	4 组 / 4600 次

第一部分的 9 组测试由脚本依次执行，每组间隔 30 秒，每组只运行一次，结果全部保留；第二部分吞吐上限测试的设计见 2.6 节。线程数指 JMeter 启动的线程总数，例如 1200-10-2 表示 1200 个线程在 10 秒内逐步启动，每个线程发送 2 次请求，共 2400 次请求。

统计全部由原始 JTL 的 elapsed 字段计算：均值、中位数、P95、P99（线性插值）；吞吐量为请求数除以首个请求开始到最后一个响应结束的时长。以“至少 80% 的请求成功且 elapsed ≤ 40 ms”作为响应时间达标条件。计算结果已与 JMeter 报告中的请求数、失败数、均值和吞吐量核对一致。

2.3 读取结果

线程数	请求数	失败	均值 ms	中位数 ms	P95 ms	P99 ms	≤40 ms 比例	吞吐量 /s
1000	2000	0	2.38	2.00	4.00	17.00	99.75%	205.21
1050	2100	0	2.14	2.00	3.00	16.00	100.00%	203.02
1100	2200	0	2.39	2.00	3.00	29.00	99.64%	226.08
1150	2300	0	2.13	2.00	3.00	16.01	99.87%	225.78
1200	2400	0	2.33	2.00	3.00	19.00	99.29%	253.94

表 1 JPA 读取测试结果（10 秒升压、每线程 2 次循环）

5 组读取共 11000 次请求全部成功，均满足达标条件。各档平均响应时间为 2.13–2.39 ms，中位数均为 2 ms，P95 不超过 4 ms，40 ms 内成功比例在 99.29% 以上。线程数从 1000 增加到 1200，响应时间没有明显上升，说明在该负载范围内读取路径远未饱和。

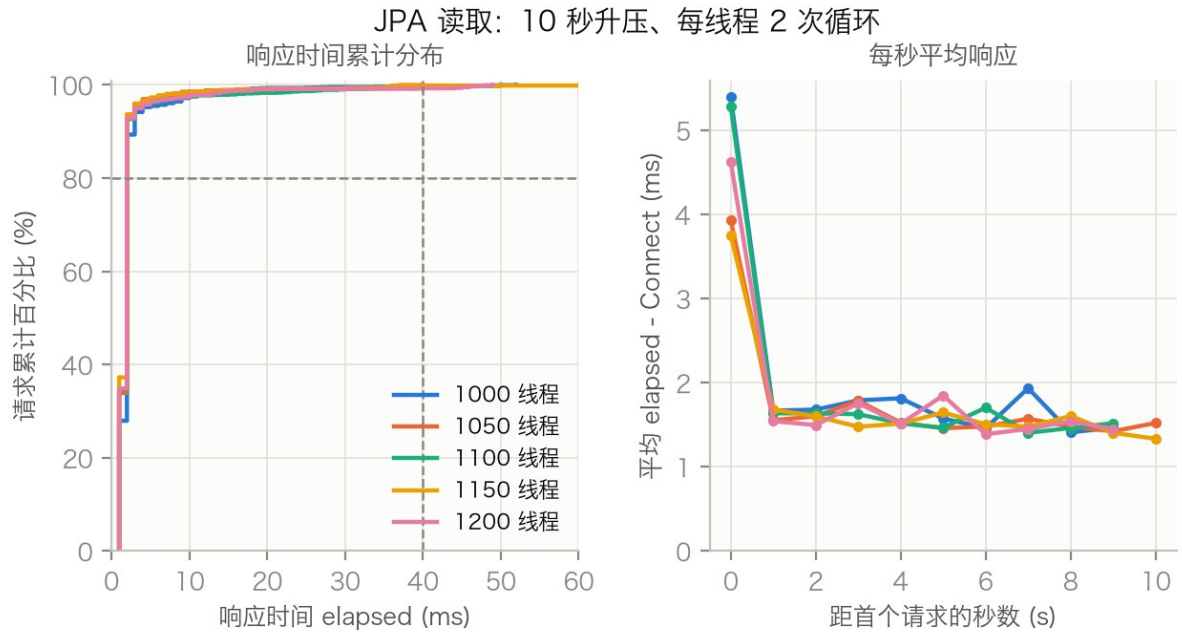


图 1 JPA 读取：左为响应时间累计分布（虚线为 40 ms 与 80%），右为按 1 秒分段的平均 elapsed - Connect

各组第 0 秒的平均响应（3.7–5.4 ms）高于之后各秒（约 1.5 ms）。这一秒内 JMeter 集中创建线程、建立 TCP 连接，压测机 CPU 出现瞬时峰值（见 2.5 节），之后曲线基本平稳。

2.4 写入结果与读写比较

线程数	请求数	失败	均值 ms	中位数 ms	P95 ms	P99 ms	≤40 ms 比例	吞吐量 /s
1000	1000	0	9.56	7.00	11.00	153.01	98.00%	101.14
1100	1100	0	6.73	6.00	8.00	28.04	99.36%	112.12
1200	1200	0	6.86	6.00	9.00	42.00	98.67%	126.50
1300	1300	0	6.79	6.00	9.00	42.00	98.77%	126.46

表 2 JPA 写入测试结果（10 秒升压、每线程 1 次循环）

4 组写入共 4600 次请求全部返回 201，均满足达标条件。1100–1300 线程的平均响应约 6.7–6.9 ms，中位数 6 ms，P95 为 8–9 ms。1000 线程组的均值（9.56 ms）和 P99（153.01 ms）偏高，图 2 显示超出部分集中在第 0 秒；该组是读取测试之后的第一组写入，推测与插入路径首次执行时的预热有关。

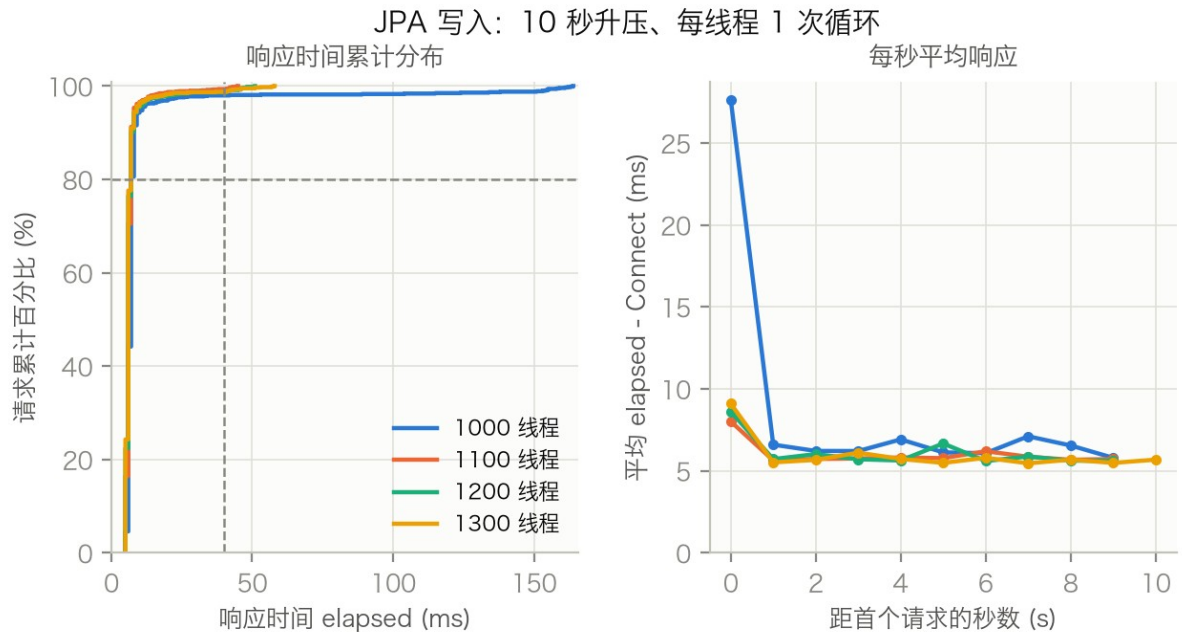


图 2 JPA 写入：左为响应时间累计分布，右为按 1 秒分段的平均 elapsed - Connect

线程数	读取均值 ms	写入均值 ms	读取中位数 ms	写入中位数 ms	写/读 均值比
1000	2.38	9.56	2.00	7.00	4.0
1100	2.39	6.73	2.00	6.00	2.8
1200	2.33	6.86	2.00	6.00	2.9

表 3 读写共有线程档位下的响应时间对比

在读写共有的 1000、1100、1200 线程档位下，写入的平均响应时间是读取的 2.8–4.0 倍，中位数为 6 ms 对 2 ms。读取只需按主键查询一行；写入需要构造实体、执行 INSERT 并提交事务，MySQL 还要写入 redo log，因此单次请求耗时更长。2.5 节中数据库节点的 CPU 峰值也印证了写入对数据库的压力明显大于读取。

2.5 运行监控

按指导书要求查询 spring:jvm_threads_live_threads、node:cpu_usage_percent 和 node:memory_usage_percent，以每组首个请求前 15 秒到最后一个响应后 30 秒为窗口、2 秒为步长导出数据，每组一个 JSON 文件。

node:cpu_usage_percent 记录规则基于 rate[5m] 计算、每 30 秒评估一次，对只持续约 10 秒的压测几乎没有反应。为观察压测期间的变化，另外导出了基于 irate[10s] 的瞬时 CPU 使用率作为补充，已在 JSON 中单独标注。spring:jvm_threads_live_threads 同样是每 30 秒计算一次的记录规则，数值变化会滞后最多 30 秒，因此另外导出了应用每 2 秒被抓取一次的原始指标 jvm_threads_live_threads，图和表中的 JVM 线程数均以原始指标为准。

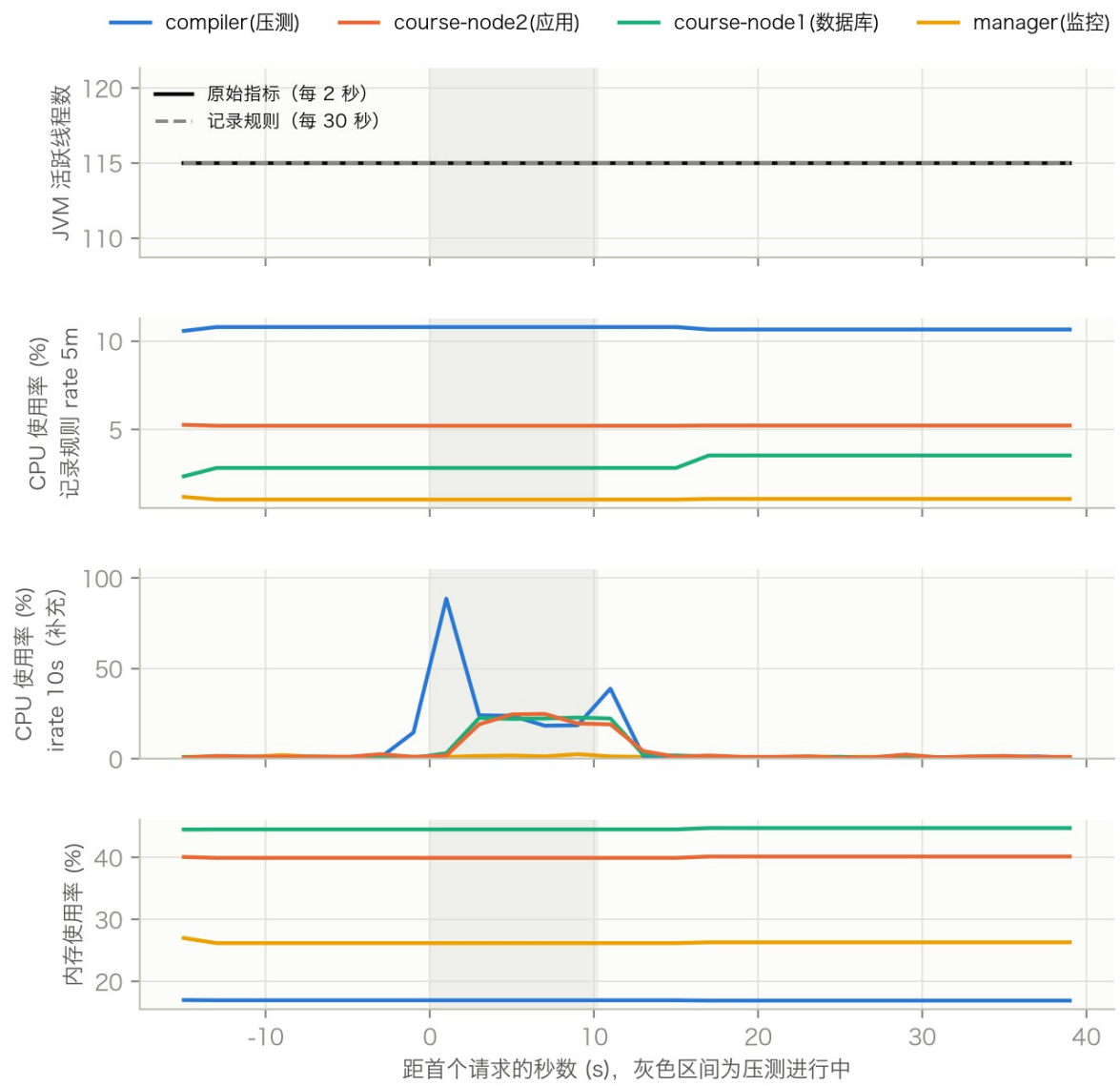


图 3 1300 线程写入期间的 JVM 活跃线程（原始指标与记录规则）、各节点 CPU（记录规则与 10 秒瞬时值）及内存使用率

测试	压测机 compiler	应用 course-node2	数据库 course-node1	JVM 活跃线程
读取 5 组	77.5%–93.3%	18.2%–28.7%	4.5%–5.5%	115
写入 4 组	54.2%–88.5%	24.7%–53.0%	19.3%–23.2%	115

表 4 压测期间各节点瞬时 CPU 使用率峰值范围（irate 10s）与 JVM 活跃线程数

压测期间 JVM 活跃线程始终为 115。这里的“1000–1300 线程”是 JMeter 在压测机上创建的虚拟用户，并不是应用的线程；这些线程在 10 秒内陆续启动、每个只发 1–2 次请求，按每秒约 130–250 个请求、每个请求 2–10 ms 估算，应用同一时刻平均只有约 1 个请求在处理。应用启动时 Tomcat 已预先创建 100 个空闲工作线程，加上 JVM 自身的后台线程共 115 个，足以应付该负载，因此没有新增线程。应用节点 CPU 峰值不超过约 53%，数据库节点在写入时的 CPU 峰值（约 19%–23%）是读取时（约 5%）的 4 倍左右。负载最重的是压测机本身：JMeter 在开始阶段集中创建上千个线程，CPU 瞬时峰值达到 54%–93%。各节点内存使用率在测试前后基本不变。

2.6 吞吐上限测试

前 9 组的吞吐量（读取 203.02–253.94/s，写入 101.14–126.50/s）约等于“线程数 × 循环次数 ÷ 10 秒”，也就是 JMeter 发出请求的速率，而响应时间始终在毫秒级，说明系统还没有被压满。为测出读写各自能承受的最大吞吐量，另设计一组测试：线程数取 20、50、100、200，升压 5 秒，每个线程收到响应后立即发送下一个请求，此时吞吐量由系统处理能力决定。

写入每组 10000 次请求；读取最初也按每组 10000 次执行，但读取速度快，每组只持续 7–8 秒，扣除 5 秒升压后稳态不足 2 秒，因此改为每组 30000 次重新执行（第一次的结果保留在压测机上，未纳入统计）。稳态吞吐量取升压结束后到最后 1 秒之前完成的请求数除以该时长，排除升压和收尾阶段的影响。

线程数	循环	请求数	失败	稳态吞吐 /s	均值 ms	中位数 ms	P95 ms	P99 ms	≤40 ms 比例
20	1500	30000	0	1566.90	10.31	9.00	24.00	37.00	99.25%
50	600	30000	0	1525.70	25.72	23.00	57.00	77.00	82.29%
100	300	30000	0	1499.29	51.51	47.00	115.00	152.00	43.91%
200	150	30000	0	1474.21	102.86	99.00	216.00	270.00	17.77%

表 5 JPA 读取吞吐上限测试（5 秒升压，每组 30000 次请求）

线程数	循环	请求数	失败	稳态吞吐 /s	均值 ms	中位数 ms	P95 ms	P99 ms	≤40 ms 比例
20	500	10000	0	640.73	26.76	22.00	59.00	99.01	86.26%
50	200	10000	0	803.10	42.25	38.00	90.00	125.00	53.40%
100	100	10000	0	820.07	79.22	76.00	159.00	200.00	22.40%
200	50	10000	0	786.95	159.73	156.00	310.05	387.00	9.17%

表 6 JPA 写入吞吐上限测试（5 秒升压，每组 10000 次请求）

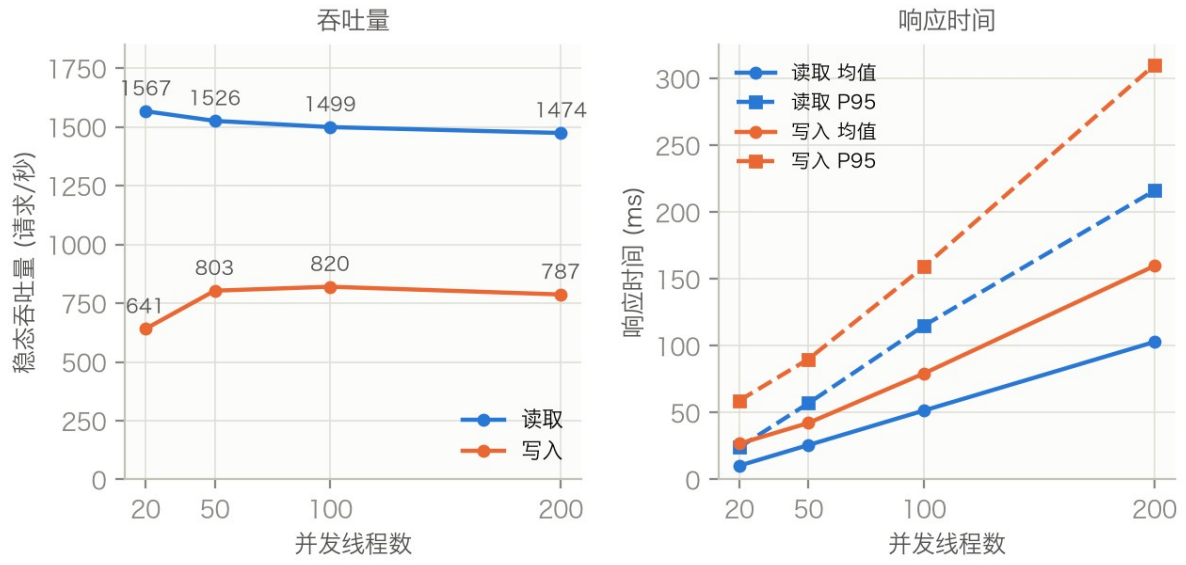


图 4 吞吐上限测试：左为稳态吞吐量，右为平均与 P95 响应时间随线程数的变化

读取在 20 线程时稳态吞吐量已达 1566.90 次/秒，继续增加线程，吞吐量不再上升（1474–1567 次/秒），平均响应时间则从 10.31 ms 增加到 102.86 ms，线程数翻倍时响应时间也约翻倍。写入在 20 线程时为 640.73 次/秒，50 线程以上稳定在 787–820 次/秒，平均响应从 26.76 ms 增加到 159.73 ms。吞吐量不变而响应时间随并发成比例增长，是系统达到饱和、多出的请求在排队的典型表现。17 组共 175600 次请求无一失败，说明饱和时应用只是变慢，没有出错。

测试	线程数	应用 course-node2	数据库 course-node1	压测机 compiler	JVM 活跃线程
读取	20	97.1%	29.5%	29.5%	115
读取	50	97.5%	27.0%	31.8%	115
读取	100	97.6%	28.6%	29.8%	115→116
读取	200	97.4%	27.3%	24.3%	116→216
写入	20	99.1%	63.2%	20.3%	134
写入	50	95.9%	69.5%	21.0%	134
写入	100	97.9%	70.8%	24.2%	134
写入	200	99.2%	66.9%	22.9%	134→199

表 7 吞吐上限测试稳态阶段各节点平均 CPU 使用率（irate 10s）与压测期间 JVM 活跃线程数

监控数据给出了瓶颈所在：读写测试的稳态阶段，应用节点 CPU 都在约 96%–99%，而压测机只有 20%–32%，可以排除压测机成为瓶颈的可能。数据库节点在读取时约 27%–30%，写入时升到 63%–71%，虽然未满，但写入时数据库的负担明显更重。因此在本环境下，读写吞吐量都受限于应用服务器的 2 核 CPU；写入的上限约为读取的 0.53 倍，原因是每次写入在应用端要做参数校验、实体构造、事务管理和 INSERT，在数据库端还要写日志和提交事务，单次请求消耗的 CPU 比按主键查询多得多。

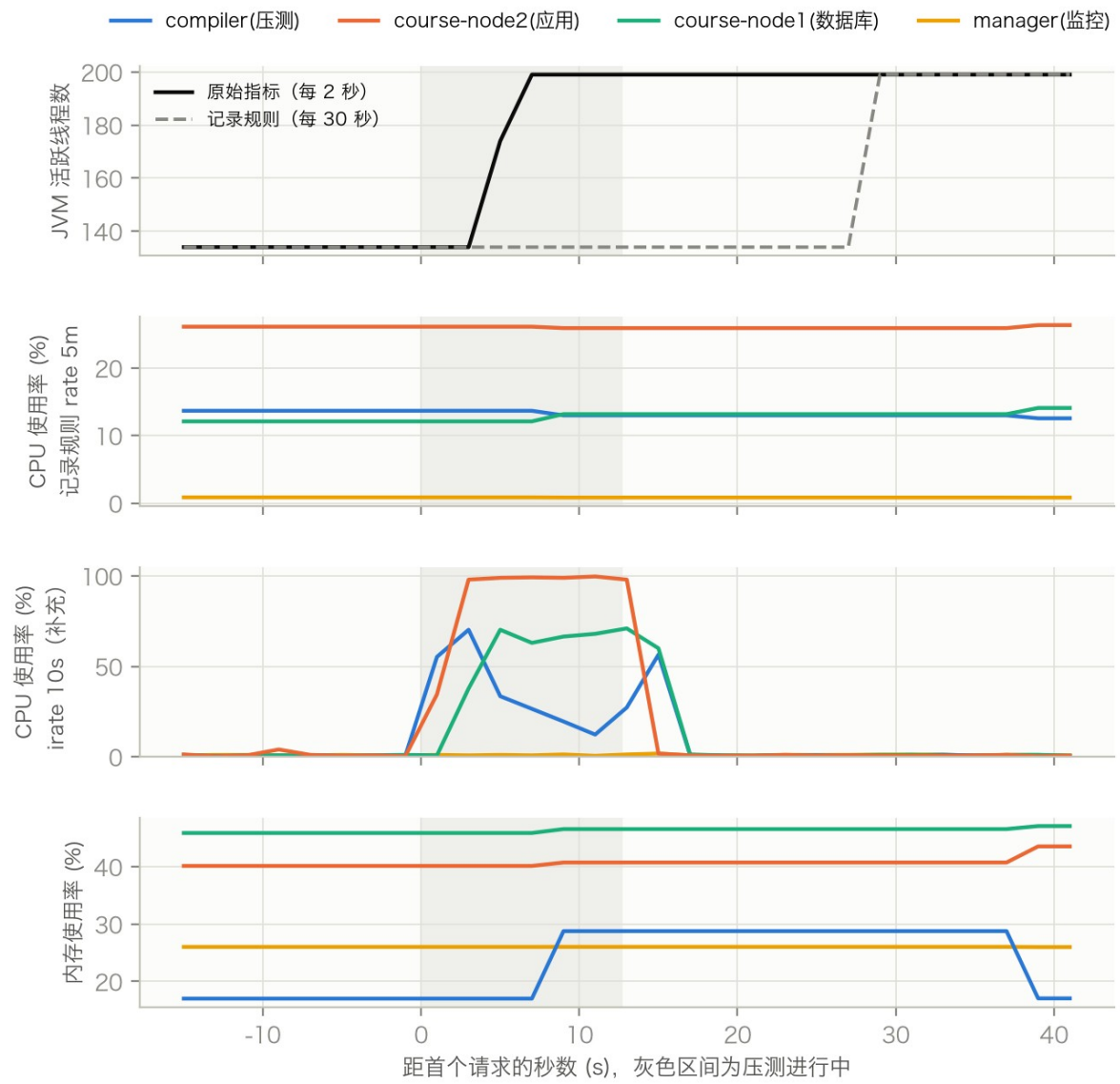


图 5 吞吐上限测试中 200 线程写入期间的 JVM 活跃线程、各节点 CPU 及内存使用率

与第一部分不同，吞吐上限测试中的线程连续发送请求，应用 CPU 已满，请求在服务器端排队。JVM 活跃线程只在 200 线程的两组中明显增加：读取由 116 增至 216，写入由 134 增至 199，都发生在升压结束（约第 5–7 秒）、200 个线程全部开始发送请求之后，说明此时同时处理的请求超过了预先创建的空闲线程，Tomcat 开始新建工作线程。20–100 线程各组基本没有增加；写入这几组的 134 是更早的测试留下的线程。监控数据显示，多余的线程在空闲约 1 分钟后被回收，回落到 115。图 5 中虚线为指导书要求的记录规则，其上升比原始指标晚 20 多秒，这就是记录规则 30 秒计算一次造成的滞后。

3 总结

本实验完成了 productdemoaop 的编译打包、Swarm 部署、Prometheus 监控配置和 JMeter 压力测试。所提交 17 组共 175600 次请求的断言失败为 0。第一部分的 9 组（15600 次请求）全部满足“80% 请

求在 40 ms 内成功返回”的条件，在相同线程档位下 JPA 写入的平均响应时间约为读取的 2.8–4.0 倍。吞吐上限测试表明，读取最大约 1474–1567 次/秒，写入约 787–820 次/秒，写入上限约为读取的 0.53 倍；两者的瓶颈都是应用节点的 CPU，写入时数据库节点的 CPU 负担约为读取时的 2.4 倍。原始请求数据、监控记录、JMeter 报告和过程截图均已保存在提交材料中。

程序地址：https://github.com/xmu-work/javaEE_labs.git（lab3 目录）